

Optimizing Large Language Models for Network Intrusion Detection Systems

By

Mohammed S Ahmad

A Thesis

Submitted to the Faculty of Graduate Studies
through the School of Computer Science
in Partial Fulfillment of the Requirements for
the Degree of Master of Science
at the University of Windsor

Windsor, Ontario, Canada

2025

©2025 Mohammed S Ahmad

Optimizing Large Language Models for Network Intrusion Detection Systems

by

Mohammed S Ahmad

APPROVED BY:

M. Hassanzadeh
Department of Electrical and Computer Engineering

D. Wu
School of Computer Science

S. Samet, Advisor
School of Computer Science

January 27, 2025

DECLARATION OF ORIGINALITY

I hereby certify that I am the sole author of this thesis and that no part of this thesis has been published or submitted for publication.

I certify that, to the best of my knowledge, my thesis does not infringe upon anyone's copyright nor violate any proprietary rights and that any ideas, techniques, quotations, or any other material from the work of other people included in my thesis, published or otherwise, are fully acknowledged in accordance with the standard referencing practices. Furthermore, to the extent that I have included copyrighted material that surpasses the bounds of fair dealing within the meaning of the Canada Copyright Act, I certify that I have obtained a written permission from the copyright owner(s) to include such material(s) in my thesis and have included copies of such copyright clearances to my appendix.

I declare that this is a true copy of my thesis, including any final revisions, as approved by my thesis committee and the Graduate Studies office, and that this thesis has not been submitted for a higher degree to any other University or Institution.

ABSTRACT

Cybersecurity has played a significant role in the development of modern computer systems. Without the use of robust cybersecurity systems, software solutions deployed at the public level would be very vulnerable to attacks by malicious parties. A particular application of cybersecurity is for the detection of anomalies in networks. These systems, named network Intrusion Detection Systems (IDS), constantly need to evolve along with the changing landscape of novel threats. These are designed to protect our networks against various malicious attacks orchestrated by hackers and other agents. Over the decades, these systems have kept up with the adoption of newer technologies that have developed over the years. Hence, AI has played an important role in the improvement of IDS in the last decade. With the advancement of machine learning techniques and the development and wide scale adoption of large language model (LLM) systems, this has led to the need to adopt LLMs to enhance existing IDS and enable better novel threat detection. Given the recency of the development of LLMs and particularly their use case for IDS, there is great scope for research and improvement in this area.

In this thesis, we propose a novel method to utilize LLMs for improving IDS by the use of prompt optimization in LLMs as well as the use of ensemble methods. These methods are chosen after extensive research on the current state of the domain and identifying the shortcomings and research gaps. In our method, we propose the use of ensemble methods with 3 different LLMs, such as Llama, Claude, and Gemini. These models, when used in conjunction with prompt optimization tools, demonstrate significant performance enhancements. Using the NSL-KDD dataset, we compare the results of our novel method against a baseline and highlight the resulting performance improvements. In addition to this, we also highlight the architectural advantages of our framework and how this system promotes the interchangeability of LLMs in order to keep the system updated with newer advancements with minimal effort. These results indicate the ability of the system to overcome previous limitations, making it a robust solution for intrusion detection.

DEDICATION

I would like to dedicate this thesis to my loving parents, Shaheed Ahmad and Qudsia Sultan, who have supported me through thick and thin and enabled me to achieve my dreams. Without their steadfast support and irreplaceable advice, this journey would not have been possible. This achievement is as much theirs as it is mine. Thank you for being my constant source of strength and inspiration.

ACKNOWLEDGEMENTS

I would like to sincerely express my most profound gratitude towards my supervisor, Dr. Saeed Samet, whose input helped me immensely. With his input, I was able to look at my research with a different perspective and a more critical eye. His calm and collected demeanor throughout my time with him helped me stay calm and focused.

Secondly, I would like to express my gratitude to my thesis committee members, Dr. Dan Wu and Dr. Mohammad Hassanzadeh, for their constructive advice and insightful suggestions for my thesis. Their contributions significantly enriched the content and quality of my thesis, which I am very grateful for.

Finally, I extend humble thanks to the School of Computer Science for providing an enriching academic environment and to all those who, in various capacities, contributed to my growth and success.

TABLE OF CONTENTS

DECLARATION OF ORIGINALITY	III
ABSTRACT	IV
DEDICATION	V
ACKNOWLEDGEMENTS	VI
LIST OF TABLES	IX
LIST OF FIGURES	X
1 Introduction	1
1.1 Cybersecurity systems	1
1.2 Significance of rapid improvement of cybersecurity systems	2
1.3 Application of artificial intelligence in cybersecurity	3
1.4 LLMs and their potential in cybersecurity	4
1.5 Thesis Contributions	5
1.6 Thesis Outline	6
2 Relevant Background	8
2.1 Network Intrusion Detection Systems	8
2.2 Transformers	10
2.3 Large Language Models	12
3 Related Works	14
3.1 LLMs for performance improvements	14
3.2 LLMs for explainability improvements	17
3.3 LLMs for overall improvement of LLMs	19
3.4 LLM Ensemble methods	21
3.4.1 LLM-Blender: Ensembling Large Language Models with Pair-wise Ranking and Generative Fusion	21
3.4.2 LLM-Ensemble: Optimal Large Language Model Ensemble Method for E-commerce Product Attribute Value Extraction	22
4 Methodology, Results & Discussion	24
4.1 Our approach	24
4.2 Tools Involved	25
4.2.1 DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines	25
4.2.2 Final framework	26
4.3 Dataset	28
4.4 Evaluation Metrics	30

4.5	Experimental Results	31
4.5.1	Discussion	33
5	Conclusion and Future Work	35
5.1	Future work	36
	REFERENCES	37
	VITA AUCTORIS	42

LIST OF TABLES

3.3.1	Summary of recent research in LLM applications for anomaly detection systems	23
4.3.1	Attributes of the NSL-KDD dataset [24]	29
4.5.1	Results from experiments to evaluate proposed methodology on NSL-KDD dataset	32
4.5.2	Comparison with baseline research paper	33

LIST OF FIGURES

1.4.1	Training procedure of BERT [9]	5
2.1.1	Anomaly-Based Intrusion Detection System [27]	9
2.2.1	Encoder Decoder Transformer Architecture [25]	11
2.3.1	Example of tokenization	12
3.1.1	Framework of LogGPT [21]	16
3.3.1	Proposed method by Naito et al.[19]	20
4.1.1	LLM Ensemble Framework	25

CHAPTER 1

Introduction

1.1 Cybersecurity systems

Since the advent of computer systems, the development of cybersecurity systems has been crucial to the development of computers. Without proper security systems in place, users' private data can be accessed and manipulated by malicious parties. In order to prevent these types of attacks, robust cybersecurity systems need to be developed. Today, cybersecurity systems play a fundamental role in our increasingly digital world, protecting the vast ecosystem of interconnected devices and networks that power modern society. These protect software systems at all levels. These include:

1. Physical Level: These protect the actual hardware and infrastructure that houses our data and systems. Examples include biometric access controls, security cameras, key access, etc.
2. Network Level: At this level, security systems protect data as it moves between devices and systems. This includes firewalls, Intrusion Detection Systems (IDS), Virtual Private Networks (VPN), etc., which monitor network traffic for suspicious patterns. These may also include machine learning algorithms, which can be used for the detection of exploits such as zero-day exploits.
3. Application Level: At this level, security systems protect specific software applications and services. Examples would include Web Application Firewalls (WAF), input validation tools, authentication systems, etc.

1.2 Significance of rapid improvement of cybersecurity systems

Due to the constantly changing landscape of security attacks, cybersecurity systems have to keep iterating and evolving in order to be effective against newer types of attacks. For instance, when security systems became good at detecting traditional malware signatures, attackers developed polymorphic malware that can change its code to avoid detection [6]. Continuous threat exposure and the increasing frequency of cyber-attacks necessitate a proactive approach to cybersecurity, with organizations investing in advanced threat detection and response systems. Moreover, with global costs projected to reach \$10.5 trillion annually by 2025 [17], the financial impact of cybersecurity cannot be understated.

Computing power continues to increase exponentially, which affects security in two critical ways. First, attacks that were once computationally infeasible become possible – what might have taken years to crack can now be done in hours. Second, defensive systems can now employ more sophisticated protection methods, like advanced machine learning algorithms that can detect subtle patterns indicating an attack. This means that existing software and hardware are open to new vulnerabilities. The Spectre and Meltdown vulnerabilities discovered in modern processors required fundamental changes to how operating systems handle memory protection [1]. Moreover, the interconnected nature of modern systems means that changes in one area often necessitate updates in others. For example, when a web browser updates its security protocols, web applications must update to remain compatible, and when cloud providers enhance their security features, client systems need updating to take advantage of these improvements. This illustrates how even seemingly secure systems can harbor unknown weaknesses that, when discovered, demand immediate updates.

1.3 Application of artificial intelligence in cybersecurity

Artificial Intelligence (AI) has grown in prominence in the past decade. With the advance of capable hardware and advanced machine learning models, traditional software has been augmented and improved rapidly with the use of AI. As such, cybersecurity systems also underwent a transformation, with AI being used heavily to improve performance. Machine learning algorithms analyze vast amounts of data to identify patterns and detect unusual activities that may indicate a security breach. The integration of machine learning into cybersecurity has fundamentally changed how we approach network protection.

A great deal of research has been done to integrate AI into cybersecurity systems. One of the most significant improvements has been in the area of intrusion detection systems (IDS), where machine learning algorithms have largely superseded traditional rule-based systems. Machine learning techniques such as Random Forests, K-nearest neighbor (KNN), and hidden Markov models were utilized for intrusion detection systems by Zhang et al. [31], Vishwakarma et al. [26], and Ariu et al. [7], respectively, amongst others. Similarly, anomaly detection systems were enhanced by the use of neural network models [5][28][4] and adaptive boosting [29], amongst others. These architectures have demonstrated remarkable capabilities in identifying both known attack patterns and discovering novel threats that conventional systems might miss.

One of the most challenging aspects of cybersecurity is dealing with zero-day attacks and previously unknown threats. Machine learning has made significant strides in this area through unsupervised learning techniques that can identify anomalous patterns without requiring pre-labeled training data. This capability is crucial in modern cybersecurity, where new attack vectors emerge constantly. Perhaps equally important is their ability to operate in real-time environments, making them practical for protecting active networks. Machine learning systems have also proven particularly adept at handling imbalanced datasets, where some types of attacks are rare but potentially very dangerous.

While these advancements greatly improved the performance of various cybersecurity systems, it's important to acknowledge the challenges that remain. These include the need for high-quality training data, the computational resources required for some more complex models, and the potential vulnerability to adversarial attacks specifically designed to fool machine learning systems. As such, this required us to use newer, more advanced models for further development of cybersecurity systems.

1.4 LLMs and their potential in cybersecurity

While AI models had brought substantial improvements to the state of cybersecurity, the improvements were stagnating. This was when transformers and Large Language Models revolutionized the scope of AI and, consequently, cybersecurity.

The paper, titled “Attention Is All You Need” [25], introduced a new type of deep learning model called transformers, which utilized an attention mechanism. Attention is a mechanism that allows neural networks to focus on specific parts of their input, which, when combined with an encoder-decoder mechanism, allows it to capture long-range dependencies. This was a big shift since existing models at the time could only capture context in a small window. This led to the development of Large Language Models (LLMs), which could perform Natural Language Processing (NLP) tasks such as language translation, question answering, and sentiment analysis with a high degree of accuracy and reliability.

Building the foundation of this paper, the paper introducing Bidirectional Encoder Representations from Transformers (BERT) was published in 2019 [9]. Unlike previous models, which processed text either left-to-right or right-to-left, BERT was designed to pre-train deep bidirectional representations by joint conditioning on both left and right context in all layers (see Figure 1.4.1). The model was pre-trained on a massive corpus consisting of the BooksCorpus (800M words) and English Wikipedia (2,500M words). The researchers demonstrate BERT’s effectiveness across a variety of NLP tasks, marking a great shift in the research of LLMs.

Following this, a great deal of research was done on Large Language Models in

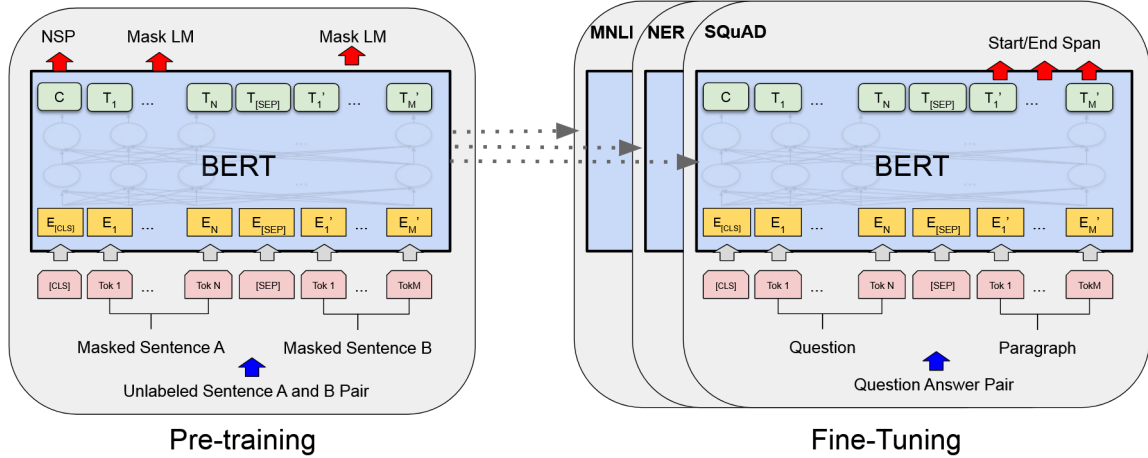


Fig. 1.4.1: Training procedure of BERT [9]

the upcoming years. The release of ChatGPT by OpenAI sent waves across the industry. Demonstrating unseen capabilities in terms of natural language processing tasks, these models were a great breakthrough for machine learning models entering the mainstream. After this, research in the areas expanded even more, with better performing models every few months as well as exploration of these models for a variety of domains, including cybersecurity.

Given the potential of LLMs for a multitude of tasks across all domains of computer science, these were also studied in the context of cybersecurity and how LLMs can be used to enhance cybersecurity systems. In this thesis, we present a comprehensive review of the state of the domain and the research done in the area, as well as discuss potential shortcomings in current research, which could be further explored in the future.

1.5 Thesis Contributions

This thesis addresses the intrusion detection and anomaly detection problem in intrusion detection systems. The proposed approach optimizes the performance of LLMs on a network intrusion dataset by the use of prompt abstraction, optimization and ensemble methods such as max-voting, which is a commonly used ensemble method for other machine learning models. The use of DSPy, a prompt abstraction framework,

along with ensemble methods, helps improve the performance of LLMs for network IDS while ensuring the adaptability of the system. The key contribution of this thesis can be summarized as:

- To the best of our knowledge, we conducted the first comprehensive evaluation of prompt abstraction and ensemble methods for LLMs in cybersecurity scenarios, particularly for network intrusion detection systems. Using popular LLM models, both open-sourced and closed-source, such as Meta’s LLama 3, Google’s Gemini 1.5 Flash, Anthropic’s Claude 3.5 Haiku, etc., we demonstrate the importance of prompt optimization for LLMs and how, with proper optimization, input prompt engineering methods can match fine-tuning methods while greatly enhancing the adaptability and updateability of cybersecurity systems.
- This thesis proposes a novel approach to obtain optimal results from LLMs. This involves optimizing the prompt according to the scenario and combining the prompt-optimized results of many LLMs using ensemble methods in order to improve the performance of the system.

1.6 Thesis Outline

The upcoming chapters in this thesis are organized as follows:

Chapter 2 covers the background of this thesis. This covers the history of the field and the backbone of the tools and technology for the completion of this study. This includes understanding the basics of network intrusion detection systems, the underlying architecture that powers LLMs, its workings as well as the state of modern LLM technology.

Chapter 3 deals with the existing research in this area. This chapter will cover existing research at the intersection of artificial intelligence, more specifically LLMs, and cybersecurity, particularly intrusion detection systems (IDS). We also note the main limitations of these works and how these inform the direction of this thesis.

Chapter 4 outlines the exact methodology proposed in this study. This includes

the tools involved, the intricacies of their implementation, the dataset involved in the training and testing purposes, and the evaluation metrics used to evaluate the results of our experiments. It also deals with the results of the experiments. Here, we showcase the obtained results and conduct a discussion & analysis to highlight the main takeaways of our research as well as talk about the difficulties involved in implementing this framework.

Chapter 5 concludes this thesis. It provides a summary of the study and its results and talks about its limitations and improvements that can be explored in future research.

CHAPTER 2

Relevant Background

In this chapter, we highlight the key concepts associated with the undertaking of this thesis. This includes a brief introduction to network intrusion detection systems (IDS), large language models (LLM), their workings as well as the current state of LLM technology.

2.1 Network Intrusion Detection Systems

Intrusion detection systems (IDS) are a network security technology that detects vulnerabilities that can be exploited against a target application or computer. These can only detect an exploit or attack in the system; they cannot automatically take action to prevent the exploit from causing harm. These systems work by analyzing a copy of the data traffic stream and detecting anomalies. This ensures that the IDS does not affect the performance of the network.

When these systems were first developed, they were slower and more prone to bypass. However, over the decades, these systems have evolved to be faster and detect a larger variety of attacks. As a result, these systems could be specialized on a specific part of the data stream. These are mainly network-based, host-based, protocol-based, application protocol-based and hybrid approaches. However, for this thesis, we will only focus on network-based intrusion detection systems (NIDS).

NIDS are used to detect suspicious activity in the network before any damage is done to the network. They work by detecting deviations from normal activity. This is done by monitoring all network data, including packet data and other network

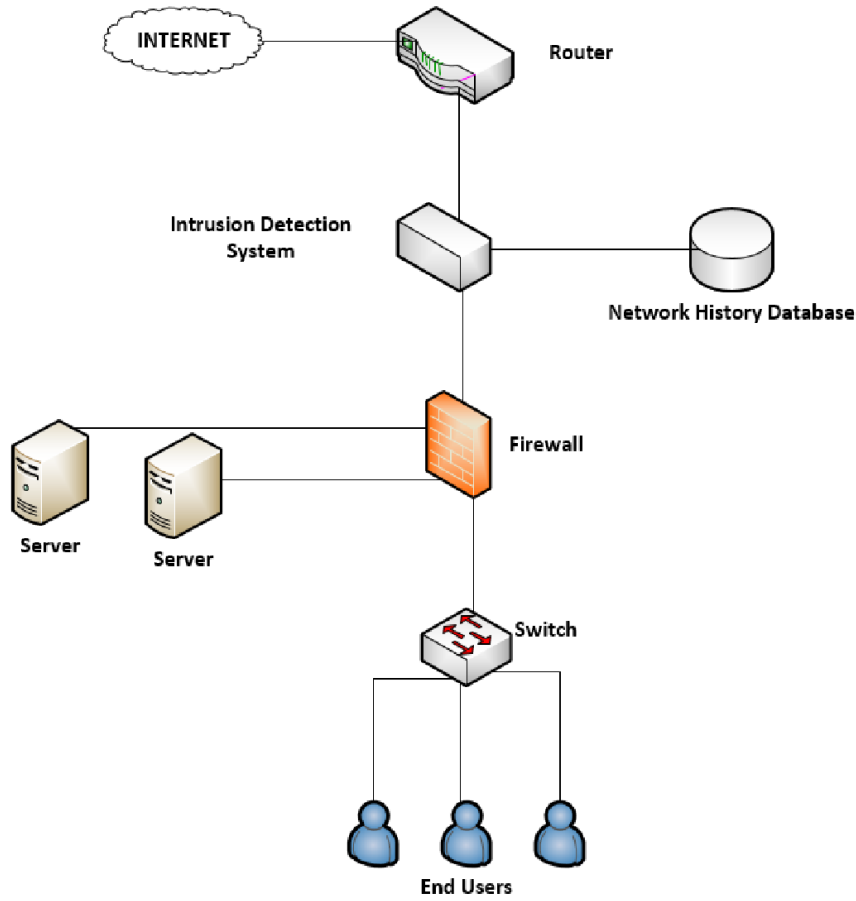


Fig. 2.1.1: Anomaly-Based Intrusion Detection System [27]

metadata such as duration of packets, error rate, etc. They can detect events such as DNS poisonings, Christmas tree scans, etc. There is also a subgroup of IDS detection methods, namely signature-based and anomaly-based. Signature-based IDS look for specific patterns that form an attack signature and match these against the attack signatures of known malicious signatures. The basic working of an intrusion detection system is shown in Figure 2.1.1

With the rise of machine learning methods for various applications of computer science, anomaly-based intrusion detection systems were developed. Anomaly-based IDS is a relatively newer technology that utilizes machine learning to analyze large amounts of network data and traffic. These IDS often catch new cyberattacks that

might evade older signature-based systems. Since these systems build a defined model of normal activity and subsequently detect anomalous activity, these are prone to false positives. Due to its recency and error-prone behavior, there is a great scope for the improvement of these systems. With the introduction of the transformer architecture in 2017, these systems opened a new range of possibilities for improvements.

2.2 Transformers

The paper “Attention is All You Need” marked a pivotal moment in the field of natural language processing by introducing the Transformer architecture. Unlike previous architectures that relied heavily on recurrent neural networks (RNNs) and convolutional neural networks (CNNs), the Transformer instead placed its entire focus on attention mechanisms. This radical departure from traditional approaches demonstrated that attention mechanisms alone could handle the complex relationships between input and output sequences.

The main innovation of the Transformer lies in its attention mechanism, particularly the multiheaded self-attention component (see Figure 2.2.1). This mechanism allowed the model to process input sequences by weighing the importance of different elements, enabling it to capture both local and long-range dependencies within the data. The Transformer’s architecture followed an encoder-decoder structure. Here, both the encoder and decoder are composed of stacks of identical layers, each containing two main components: the attention mechanism and a feedforward neural network. The encoder used self-attention layers, where all keys, values, and queries came from the same source, allowing each position to attend to all positions in the previous layer. The decoder, while similar, included an additional attention layer that helped it to attend to relevant parts of the encoded input. A key feature was the multi-head attention, which allowed the model to jointly attend to information from different representation subspaces at different positions, greatly enhancing the model’s capability to capture various aspects of the input sequence.

The authors demonstrated the Transformer’s effectiveness through extensive ex-

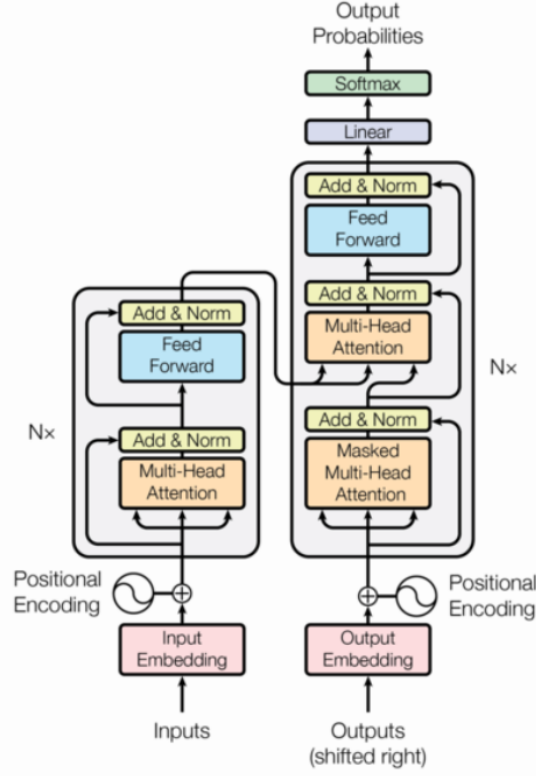


Fig. 2.2.1: Encoder Decoder Transformer Architecture [25]

periments on machine translation tasks. These results were impressive because they were achieved with less training time than previous approaches, demonstrating the architecture's superior efficiency in training and inference.

The impact of the Transformer architecture extended well beyond its immediate results in machine translation. The model's ability to efficiently handle long-range dependencies, coupled with its highly parallelizable nature, made it an ideal foundation for future developments in language modeling. This breakthrough laid the groundwork for subsequent developments in language models, including the development of large-scale models that have become instrumental in various applications across computer science domains.

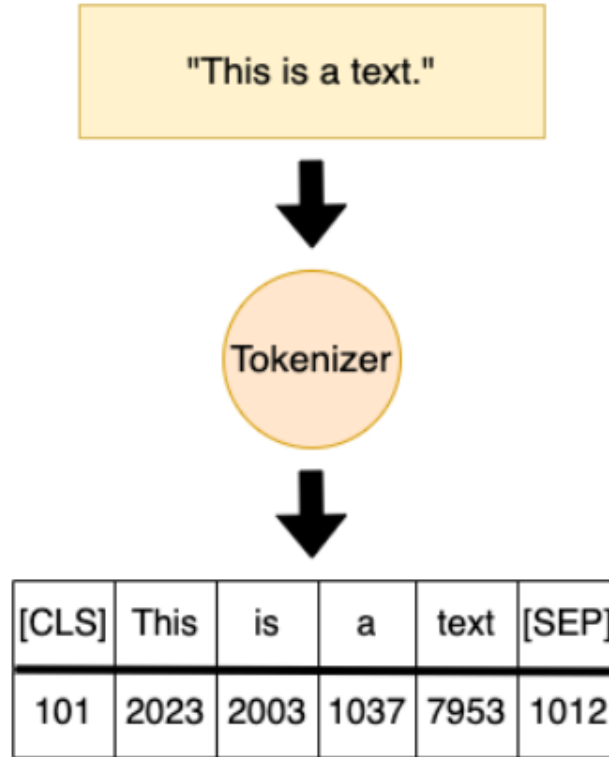


Fig. 2.3.1: Example of tokenization

2.3 Large Language Models

Following the development of the transformer architecture, further research in the area led to the development of what we now know as Large Language Models (LLM).

LLMs are deep neural language models with a massive number of parameters ranging from millions to hundreds of billions. These are trained on a large corpus of unlabelled text data. This text data is processed by a unit called a tokenizer. This component breaks text into smaller units called tokens, which are then represented as integers between 0 and $V-1$, where V is the vocabulary size. The working of a tokenizer is illustrated in Figure 2.3.1. While this is a general understanding of the workings of a tokenizer, it is important to note that different models may break text into different tokens, i.e., the tokens generated by different LLMs may not be the same. In addition to breaking down phrases into tokens, tokenizers may also add some special for special indications, such as the beginning and end of sentences,

as well as for words that are not in existing vocabulary. These tokens will then be assigned a unique integer. These integers can then be utilized to train the LLM. The most common method of training an LLM is to train the model to predict the next word given a text segment.

Since these models require a lot of text data and computational resources to train, in order to produce well-performing models, these models cannot be trained by just anyone but by the organizations with the resources to do so. As a result, in the past two years, there has been a great influx of LLMs such as OpenAI’s ChatGPT, Google, Gemini, Anthropic’s Claude, etc. There has been a lot of progress in the accessibility of these models with the release of open-source models such as Meta’s Llama models and Mistral AI’s Mixtral models, amongst many others. This is a rapidly evolving field, with radical changes being observed almost every month. As a result, there is a need to build systems that are capable of taking advantage of these advancements on the fly and not be constricted to the technology that was available at the time of the development of the system.

With the meteoric rise of LLMs, naturally, these models were being extensively employed to analyze their usefulness in a variety of domains. This included cybersecurity, where researchers applied LLMs to research improvements across various areas of cybersecurity such as education, problem-solving like Capture-The-Flag Challenges, development of security policies, etc. In this study, we will restrict the discussion to the topic of the use of LLMs for anomaly detection systems.

CHAPTER 3

Related Works

In this chapter, we explore various approaches being employed for utilizing LLMs in the domain of cybersecurity, particularly anomaly detection systems. We also take a look at the research done on the relatively new topic of LLM ensembles.

3.1 LLMs for performance improvements

G.Lira et al.[12] proposed an approach for using LLMs for Intrusion Detection Systems (IDS), particularly network IDS (NIDS), exploring their capacity to process raw logs from monitoring tools for the purpose of detecting and classifying attacks. But unlike other approaches that utilize prompt engineering on GPTs, they utilized the BERT [9], which was fine-tuned to be capable of identifying and classifying network attacks within text logs. This model was called BERTIDS. Using BERT allows the researchers to define a large context window which will be able to utilize a larger combination of features to identify and classify attacks. They utilized the NSL-KDD dataset, which is a commonly used dataset for the assessment of IDS techniques. Using the dataset, synthetic log data was generated, which could then be fed to LLM for fine-tuning. Upon evaluating the BERTIDS model, the researchers found that this model was outperforming most previous Neural Network approaches across most evaluation metrics, coming very close to the performance of top models in metrics such as Recall and False Positive Rate (FPR). It was able to do so while utilizing fewer features for detection. Moreover, these models are adaptive and can learn new threats as they emerge over time. Some limitations noted here are the lack of inter-

pretability and, subsequently, the lack of transparency in the decision-making process, which can lead to a lack of trust in these models by cybersecurity professionals. In addition to this, an important point noted by the researchers is the significant computational and temporal cost of training and fine-tuning these models. Overfitting is also a major challenge that needs to be addressed actively when using LLMs in IDS implementations.

J. Qi et al.[21] developed a log-based anomaly detection framework based on ChatGPT. The promise showed by LLMs in various domains, such as language understanding, machine translation, etc., prompted them to utilize them for log-based anomaly detection. This includes three steps, namely log preprocessing (transforming logs into a structured format), prompt construction (instructions for ChatGPT), and a response parser (responsible for extraction and analysis of ChatGPT output). The researchers evaluated LogGPT (see Figure 3.1.1) on two public datasets (BGL and Spirit[16]) and compared its performance against three deep learning baselines (DeepLog[10], LogAnomaly[18], and LogRobust[32]). Upon evaluation, LogGPT showed promising results in both zero-shot and few-shot settings, outperforming baseline methods in terms of F1 score, precision, and recall. They found that the prompt construction has a great impact on the output of LogGPT. It was also observed that increasing window size generally improves detection accuracy. Besides these improvements, LogGPT showed great improvements in interpretability, providing specific information about detected anomalies and potential preventive measures, unlike deep learning methods. The researchers do highlight the sensitivity of the results to prompt variations. Potential future improvements proposed by the authors include experimentation on more datasets and including a greater variety of GPTs.

Akuthota et al. [2] present a study on utilizing Large Language Models (LLMs), specifically the GPT-3.5-Turbo model, for vulnerability detection and monitoring in software code. They developed a methodology that involved scanning code snippets, identifying security vulnerabilities, and proposing potential fixes. The study used a dataset of 2,740 test cases from the OWASP Benchmark repository. The researchers implemented a function to analyze code and metadata, employing fuzzy

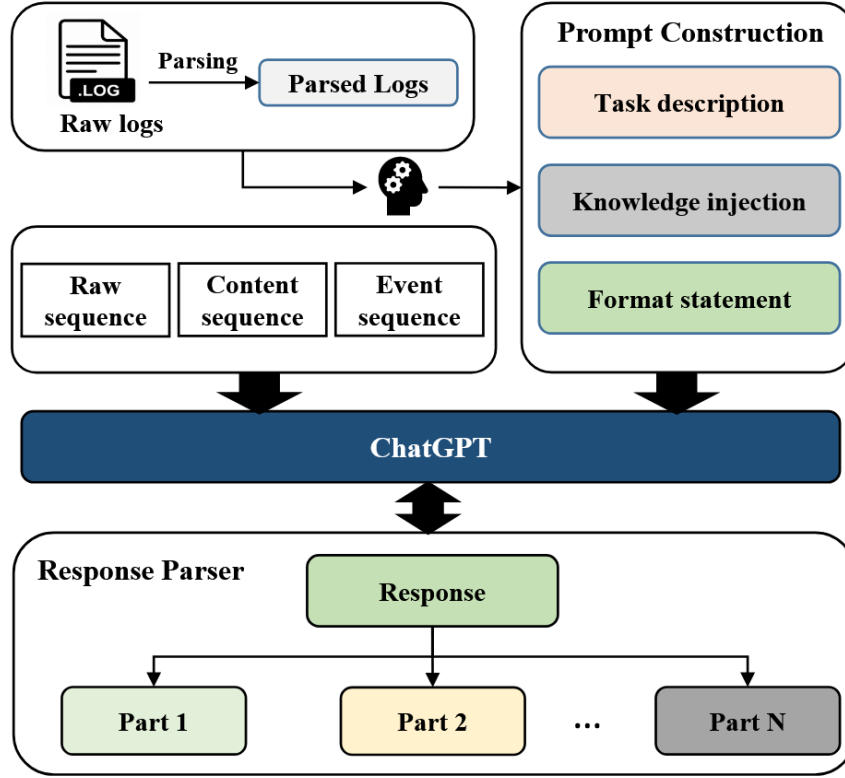


Fig. 3.1.1: Framework of LogGPT [21]

string-matching techniques to improve detection accuracy. They also integrated automated testing using the OWASP Benchmark to evaluate the effectiveness of their approach. The results showed that the GPT-3.5-Turbo model achieved an accuracy of 0.77 in detecting vulnerabilities. The model was able to identify and categorize various types of vulnerabilities, with SQL Injection, Weak Randomness, and Cross-Site Scripting (XSS) being the most frequently detected. The authors suggest that this approach can be beneficial for developers in production environments, release managers for quality assurance, and for implementing continuous monitoring systems. They acknowledge the limitations of their work, noting that LLMs trained specifically on code-based data could potentially improve results. For future work, the researchers propose converting their codebase into an open-source application, optimizing prompt instructions, investigating the impact of temperature changes on LLM output quality, and exploring the model's proficiency across multilingual datasets.

Chen et al.[8] propose a method that combines the diamond model of intru-

sion analysis with deep learning models, specifically Bidirectional Encoder Representations from Transformers (BERT)[9] and Generative Pre-trained Transformers (GPTs), to enhance intrusion analysis against cyber-attacks. They collected diverse threat reports on specific malware families and formulated threat entity extraction as a question-answering task based on the diamond model’s elements. Reports that didn’t specify the type of malware were excluded to ensure the resulting dataset was of high quality. The study compared the performance of fine-tuned DistilBERT and GPT-3 (text-davinci-003) models in extracting relevant information from the threat reports. The preprocessing involved minimal cleaning to retain as much context as possible. The researchers evaluated the models’ outputs through manual assessment by information security professionals, categorizing results as correct, incorrect, or not mentioned in the text. Over 79 randomly sampled outcomes, results showed that GPT outperformed BERT, achieving an accuracy rate of 61% compared to BERT’s 44%. The authors suggest that future work should focus on improving model accuracy by refining question formats and introducing input data restrictions.

3.2 LLMs for explainability improvements

The use of LLMs for NIDS was also explored by Ziems et al.[33] in their paper titled “Explaining Tree Model Decisions in Natural Language for Network Intrusion Detection”. They noted that while the performance of modern NIDS has improved rapidly, their explainability has taken a hit. While decision tree-based NIDS are able to identify threats with good accuracy, they are unable to provide an explanation as to which factors led to the conclusion. Since they require a user familiar with their underlying mechanics to interpret why a particular prediction was made, these explanations remain inaccessible to a non-expert audience. Hence, they propose a solution where they utilize modern LLMs like GPT-4 to generate explanations for the decisions taken by the decision tree-based NIDS. Using appropriate prompts with high-quality, relevant background knowledge, the prompted LLM is able to generate an explanation for every decision taken. They used human evaluators to validate their

results. The LLM explanations were pitted against rule-based explanations, which are the node-splitting rules used in the decision tree. Upon evaluation, the LLM explanations were found to be far better than rule-based explanations. They scored 15% higher on readability, twice as high for quality, and 70% higher on background knowledge compared to rule-based evaluations. The authors note that this method is relatively slow and expensive, and performance degrades with increasing depth of decision trees.

Ali et al.[3] introduce HuntGPT, an innovative intrusion detection system that integrates machine learning-based anomaly detection with Explainable AI (XAI) and LLMs. The system utilizes a Random Forest classifier trained on the KDD99[22] dataset for network traffic analysis and anomaly detection. HuntGPT incorporates XAI frameworks like SHAP and LIME to enhance the interpretability of the model’s decisions and employs GPT-3.5 Turbo as a conversational agent to provide explanations and insights to users. The researchers developed a comprehensive dashboard that presents detected anomalies, visualizations of the model’s inner workings, and an interactive chatbot interface. They evaluated the system’s technical accuracy using Certified Information Security Manager (CISM) certification practice exams and assessed the readability of generated responses across six metrics. Results showed that GPT-3.5 Turbo demonstrated substantial proficiency in cybersecurity knowledge, achieving success rates between 72% and 82.5% on standardized certification exams. The readability analysis indicated that the generated content was generally comprehensible for individuals with a basic college education level. These results, combined with an easy-to-use user interface, make a substantial contribution to enhancing user comprehension in cybersecurity. The study highlights the potential of integrating LLMs with XAI in cybersecurity applications, offering a balance between advanced threat detection and user-friendly explanations. The authors suggest future work should focus on incorporating real-time detection capabilities and embedding actionable AI within the chatbot to enable instant responses to security events.

3.3 LLMs for overall improvement of LLMs

Singla et al.[23] evaluated the effectiveness of LLMs in analyzing software supply chain security failures. Using a dataset of 69 failures from the Cloud Native Computing Foundation’s catalog, they developed prompts for LLMs to categorize failures across four dimensions: type of compromise, intent, nature, and impact. The study compared OpenAI’s GPT-3.5 and Google’s Bard against manual analysis. GPT-3.5 outperformed Bard, achieving an average accuracy of 68% across dimensions, compared to Bard’s 58%. GPT-3.5 performed particularly well in categorizing intent (88% accuracy) and nature (74% accuracy). Cohen’s kappa, a measure of agreement between raters that accounts for chance, was 0.87 for intent and 0.58 for nature, indicating substantial and moderate agreement, respectively, which correlated with GPT-3.5’s stronger performance in these areas. However, with lower agreement among human raters (Cohen’s kappa of 0.34 for impact), the LLMs struggled more, especially with multi-answer options. The researchers also introduced a “Lessons learned” category, where GPT-3.5 achieved an average helpfulness score of 3.83/5. The study found that LLMs perform well with detailed source articles and when manual analysts agree but struggle with complex and ambiguous cases. They suggest that future work should include fine-tuning LLMs on specialized cybersecurity data and expanding the analysis to a broader range of failures, as improvements in LLM capabilities may eventually make them useful aids in this type of analysis.

Naito et al.[19] propose a novel approach to generating attack scenarios for penetration testing and red teaming using ChatGPT in conjunction with IT asset management data and vulnerability information. The researchers aimed to address the shortage of skilled human resources for these critical cybersecurity tasks. Their method involves inputting asset management data (including device information, OS versions, and network details) and vulnerability information from sources like Microsoft and CISA into ChatGPT. The AI then generates potential attack paths with high threat levels. The workings of this system are showing in Figure 3.3.1The study demonstrated ChatGPT’s ability to identify vulnerabilities in managed devices based on

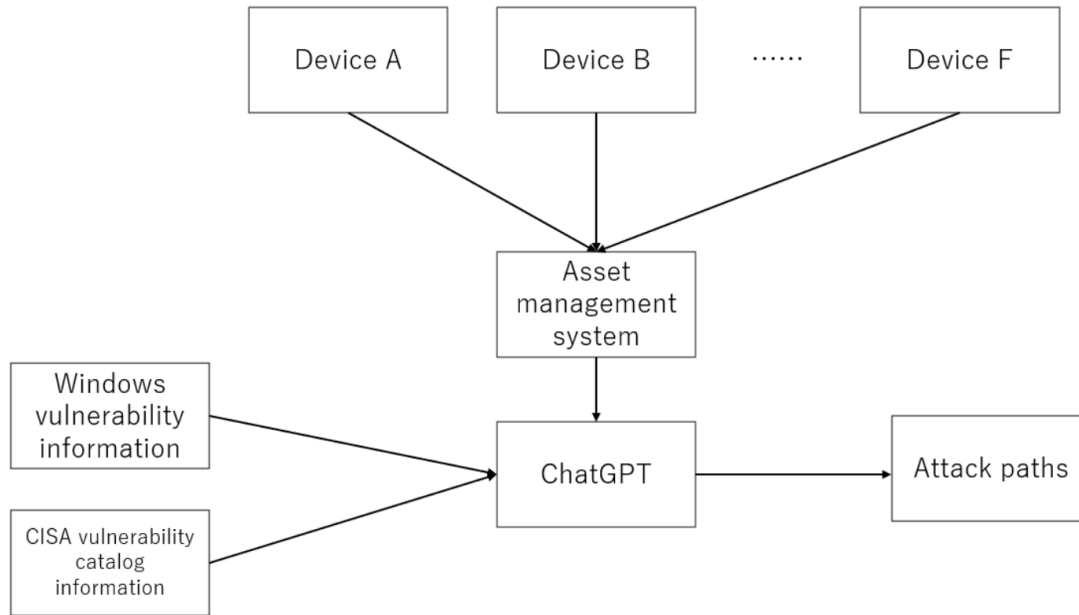


Fig. 3.3.1: Proposed method by Naito et al.[19]

their Windows update status and to map out possible attack paths for lateral movement toward critical servers. The system could determine the number of Common Vulnerability and Exposures (CVEs) applicable to each device and provide some details about these vulnerabilities. It also proposed specific attack scenarios, including the exploitation of various CVEs to move from one device to another within the network. While the research shows promise in automating and streamlining the process of identifying potential security risks, the authors acknowledge limitations such as the need for more detailed vulnerability information and validation in real-world environments. Future work suggested by the researchers includes exploring attack paths in more complex network environments, expanding automation capabilities, and developing real-time risk assessment methodologies.

A summary of the research work reviewed here is shown in Table 3.3.1

3.4 LLM Ensemble methods

In this section, we will discuss a few works not related to LLMs for cybersecurity purposes but rather the research done on the topic of LLM ensembles. This is a developing topic of research which is why the number of relevant works discussed in this section are few.

3.4.1 LLM-Blender: Ensembling Large Language Models with Pairwise Ranking and Generative Fusion

The LLM-Blender framework [14] introduces a novel approach to combining multiple Large Language Models (LLMs) by leveraging their complementary strengths while mitigating individual weaknesses. At its core are two key components: PAIR-RANKER, which uses cross-attention encoders to compare model outputs pairwise rather than scoring them individually, and GENFUSER, which fuses the top-ranked outputs to generate an improved final response. This architecture allows for a more nuanced comparison of outputs and better capture of subtle quality differences compared to traditional ranking approaches.

To evaluate their framework, the researchers created MixInstruct, a benchmark dataset with oracle rankings based on ChatGPT comparisons. Their results showed that LLM-Blender significantly outperformed both individual LLMs and baseline methods on instruction-following tasks, with particularly strong performance even when using smaller open-source models. The framework demonstrated it could help models like Llama-2-13B achieve performance competitive with larger models through effective ensembling.

The research represents an important step forward in systematic LLM combination methods, moving away from manually engineered prompts and complex fine-tuning approaches. By providing a structured way to leverage multiple models' strengths, LLM-Blender shows promise for improving response quality and consistency while potentially reducing costs through the effective use of smaller models - aspects that are particularly relevant for practical LLM deployments in various domains.

3.4.2 LLM-Ensemble: Optimal Large Language Model Ensemble Method for E-commerce Product Attribute Value Extraction

LLM-Ensemble [11] introduces a novel approach to optimize product attribute value extraction in e-commerce by combining outputs from multiple Large Language Models (LLMs). While individual LLMs have shown strong performance in attribute extraction tasks, they exhibit varying strengths and weaknesses across different product categories due to differences in their training data, architectures, and parameters. The paper proposes treating each LLM like a crowdsourcing worker and using an iterative weighted voting mechanism to leverage their complementary capabilities.

The core algorithm is based on the Dawid-Skene Model and iteratively learns optimal weights for different LLMs’ outputs to predict final attribute values. The method is theoretically proven to optimize the upper bound on error rates compared to simple majority voting approaches. The authors demonstrate their approach’s effectiveness through experiments on Walmart’s internal datasets, showing LLM-Ensemble outperforms individual state-of-the-art LLMs like GPT-4 (95.2% vs 97.9% accuracy on gender attribute extraction) as well as traditional methods like logistic regression. Beyond offline evaluations, the researchers conducted A/B testing by implementing LLM-Ensemble as a filtering layer in Walmart’s product recommendation system across over 200 categories. The tests showed statistically significant improvements in key metrics like Gross Merchandise Volume (+0.38%), Click-Through Rate (+2.16%), and Conversion Rate (+0.26%), leading to the method’s deployment in production. This practical validation demonstrates the real-world value of systematic LLM combination approaches for e-commerce applications.

Work	Results	Disadvantages
BERT-based network IDS [12]	Maximum accuracy of 98% with fine-tuning	Requires fine-tuning, reduced adaptability, potential of overfitting
Log-based anomaly detection using Chat-GPT [21]	Varied results with different prompts, maximum F1 score of 0.694	Tested with few LLMs
Code Vulnerability detection using LLMs [2]	Identify vulnerabilities with an accuracy of 77%	Prompts not optimized, GPT hyperparameters not explored
LLMs for categorization of software failures [23]	Poor performance across most categories	Prompts not optimized, Uses older LLMs
LLMs for malware attack detection [8]	Accuracy of 61% using GPT	Prompts not optimized, No restrictions on the input data
ChatGPT for attack route searching [19]	Showcased the ability to find undiscovered attack routes	Prompts not optimized
Explainability of Decision Tree-based IDS using LLMs [33]	Scored 39.5% higher than rule-based methods with high readability	Could be further improved with retrieval techniques
Explainability of Random Forest-based IDS using LLMs [3]	Demonstrated high success rate in answering questions with high readability	Uses prompt engineering

Table 3.3.1: Summary of recent research in LLM applications for anomaly detection systems

CHAPTER 4

Methodology, Results & Discussion

In this chapter, we outline our approach to the problem. This includes the overall framework, tools involved, the selected dataset, and the resulting algorithm. We then discuss the evaluation metrics used to evaluate the results of our study, followed by a presentation of the results and its subsequent comparison with a baseline paper. Lastly, we discuss the key advantages of the observed results.

4.1 Our approach

Our approach to this problem is outlined in Figure 4.1.1. Here, we use the NSL-KDD dataset, where the records are pre-processed to contain the normal and anomalous classes, making the problem a binary classification problem. Using the records from this dataset, we can form the appropriate prompt to predict the nature of the network data. This prompt contains the network data and its attributes as a part of it. This prompt is further optimized by the use of DSPy, a tool designed to optimize prompts and ensure optimal performance of the prompt across our LLMs. This is then run on three different LLMs, which provide their predictions, classifying the network data into “normal” and “anomalous” classes. The results from multiple LLMs are consolidated by the use of ensemble methods, namely max-voting, in order to evaluate the final performance of the models. This performance is compared to the performance of individual LLMs as well as the previous baseline paper.

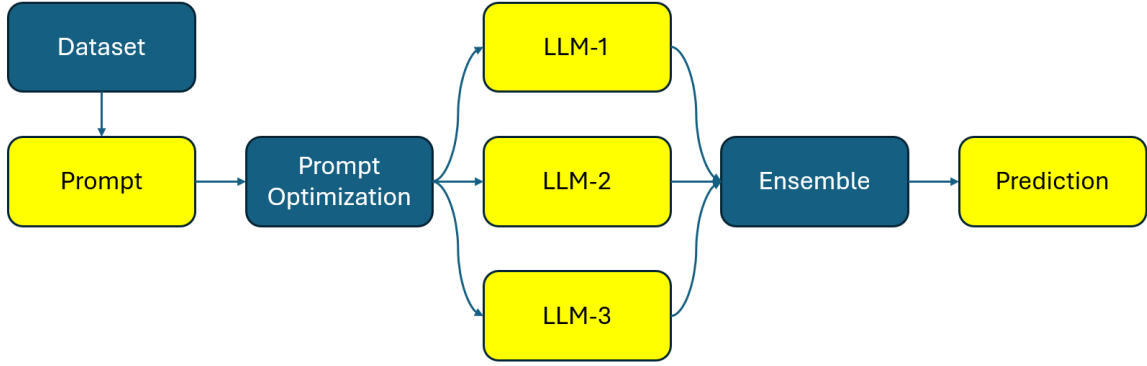


Fig. 4.1.1: LLM Ensemble Framework

In the subsequent sections, we explain the details of the components involved in the framework, the dataset as well as the evaluation metrics in detail.

4.2 Tools Involved

4.2.1 DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines

This research forms the core of our experiments. DSPy is a novel programming framework developed by Stanford researchers Khattab et al. [15] that transforms how Large Language Models (LLMs) are utilized in complex systems. Unlike traditional methods that rely on manual prompt engineering, DSPy introduces a systematic approach through three core components: signatures, modules, and optimizers (formerly known as teleprompters). These components work together to create self-improving pipelines that can enhance the performance of cybersecurity systems. In the following sections, we will explore the components of this framework:

1. **Signatures:** They form the foundation of DSPy by providing a natural language-typed declaration system for defining the expected inputs and outputs of a function. Rather than crafting specific prompts, signatures abstract the intended behavior at a higher level. This abstraction allows the system to focus on what needs to be accomplished rather than how to prompt the LLM to accomplish

it.

2. **Modules:** DSPy modules serve as parameterized components that translate various prompting techniques into modular functions. These modules can implement different reasoning strategies, such as:

- Basic Prediction: Direct transformation of input to output
- Chain of Thought: Step-by-step reasoning before making predictions
- ReAct: Interleaving reasoning and actions
- Multi-Chain Comparison: Multiple reasoning attempts with comparative analysis

The modularity allows for easy composition and modification of different approaches without rewriting prompt templates.

3. **Optimizer:** Optimizers are optimization engines that compile DSPy programs to maximize performance metrics. These work by systematically refining prompts, parameters, and program structures to maximize performance metrics like accuracy. These work by taking the DSPy modules as input, evaluating them against criteria, and iteratively optimizing parameters such as prompt templates to maximise the evaluation metric. Using this, a highly optimized prompt can be generated.

The compilation process involves three key stages: candidate generation, parameter optimization, program optimization, and implementation strategy. First, the system generates potential parameter values and demonstrations for each predictor in the program. Then, during parameter optimization, the selection of optimal parameters is done using techniques like random search or tree-structured Parzen estimators.

4.2.2 Final framework

Given the lack of applicability of this research in the cybersecurity domain, particularly the network IDS domain, we can see the potential of using these techniques in

conjunction with LLM ensemble methods to improve the performance of these systems. Also, as we found in our research in Chapter 3, most of the existing research on LLM ensembling has been done only on regression problems, not classification problems, which is why we need to use another ensembling method to suit classification problems.

Algorithm 4.2.1 Network Traffic Classification with LLM ensemble

```

1: Input: Dataset of network traffic features
2: Output: Classification predictions and metrics
3: Load dataset
4: Preprocess data: replace non-normal classes with “anomalous” for binary classification
5: for each model in model_list do
6:   Configure model API settings
7:   predictions  $\leftarrow$  empty list
8:   for each row in dataset do
9:     features  $\leftarrow$  extract network traffic features
10:    Optimize prompt using network features
11:    prediction  $\leftarrow$  model.predict(features)
12:    Add prediction to predictions list
13:    Store prediction in results
14:   end for
15:   Save model predictions for ensemble voting
16:   Calculate metrics:
17:     accuracy, precision, recall, F1-score
18:   Display evaluation metrics
19: end for
20: Ensemble Voting:
21: for each row in dataset do
22:   votes  $\leftarrow$  [llm1_pred, llm2_pred, llm3_pred]
23:   ensemble_prediction  $\leftarrow$  most common class in votes
24:   Store ensemble prediction
25: end for
26: Calculate and display ensemble vote metrics
27: Save final results for comparison

```

By combining these two approaches, we aim to develop a better system capable of exhibiting good performance while showcasing a more flexible, upgrade-friendly system. Upon using a prompt optimized to utilize the network parameters to obtain optimal results from the LLM, we combine these results from the LLMs using an

ensemble technique called max-voting. We use max-voting since this problem is a classification problem rather than a regression problem. In max-voting, a vote is taken from the LLMs in order to classify a data point as “normal” or “anomalous”. Then, upon counting the votes, the class with the majority vote is selected and presented as the final prediction of the LLM ensemble. The algorithm for this method is shown in Algorithm 4.2.1.

4.3 Dataset

The choice of dataset for this study is the **NSL-KDD** dataset [24]. This is considered a standard benchmark for intrusion detection evaluation, used to evaluate the performance of new research systems. This dataset was built as an improvement to an older benchmark dataset, the KDDcup99 dataset [22]. Performing improvements such as duplicate removal and a more uniform distribution of attack types in the train and test set led to a better-balanced dataset that was less prone to biases.

The original dataset contains a total of 125,973 training samples and 22,544 test samples. The dataset contains records of 5 different types of attacks as well as records of normal data transmission. For the purpose of our study, since we are conducting testing of our framework, we use 10% of the testing samples for our experiments. Given the complexity of our framework, we decided that this was the appropriate sample size that would produce a significant result while being experimentally viable. Moreover, in order to ensure a fair comparison with our baseline paper, we classified all attack-type records as “anomalous” and converted the problem into a binary classification, classifying network data as “normal” or “anomalous”.

The dataset consists of a total of 42 attributes, 41 describing the network data characteristics and 1 attribute describing the data as normal or anomalous. Of the other 41 attributes, 21 attributes refer to the connection itself, and 19 attributes describe the nature of connections within the same host. The packet data categorized as anomalous are further categorized into 22 intrusion attack types. These attributes are described in Table 4.3.1.

#	Attribute	#	Attribute
1	duration	22	is_guest_login
2	protocol_type	23	Count
3	service	24	srv_count
4	flag	25	serror_rate
5	src_bytes	26	srv_serror_rate
6	dst_bytes	27	rerror_rate
7	land	28	srv_rerror_rate
8	wrong_fragment	29	same_srv_rate
9	urgent	30	diff_srv_rate
10	hot	31	srv_diff_host_rate
11	num_failed_logins	32	dst_host_count
12	logged_in	33	dst_host_srv_count
13	num_compromised	34	dst_host_same_srv_rate
14	root_shell	35	dst_host_diff_srv_rate
15	su_attempted	36	dst_host_same_src_port_rate
16	num_root	37	dst_host_srv_diff_host_rate
17	num_file_creations	38	dst_host_serror_rate
18	num_shells	39	dst_host_srv_serror_rate
19	num_access_files	40	dst_host_rerror_rate
20	num_outbound_cmds	41	dst_host_srv_rerror_rate
21	is_host_login	42	class

Table 4.3.1: Attributes of the NSL-KDD dataset [24]

4.4 Evaluation Metrics

The evaluation metrics chosen for this study are accuracy, precision, recall, and F1-score. These are the standards for evaluating machine learning approaches and have been widely used in previous research. These are described as follows:

Accuracy: Accuracy is the ratio of the number of correct predictions to the total number of predictions made by the system.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

where,

TP (True Positive): Total number of packet data correctly predicted as normal

TN (True Negative): Total number of packet data correctly predicted as anomalous

FP (False Positive): Total number of anomalous packet data incorrectly predicted as normal

FN (False Negative): Total Number of normal packet data incorrectly predicted as anomalous

Precision: Precision is the ratio of the total number of positive predictions that are correct to the total number of positive predictions.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Recall: Recall is the ratio of the total number of positive predictions that are correct to the total number of predictions that are actually positive in the test dataset.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

F1-Score: Although we have calculated precision and recall, in order to evaluate the efficacy of a model on datasets with unbalanced target distributions, we must analyze both precision and recall. F1 score is defined as the harmonic mean of precision and recall. In the context of anomaly-based IDS, the F1 score would measure

how well the model can correctly identify anomalous packet data while minimizing the number of false positives (anomalous packet data incorrectly classified as normal) and false negatives (normal packet data incorrectly predicted as anomalous)

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Using these standard evaluation metrics, we can compare the performance of our ensemble method against individual LLM results as well as against the performance of the methods evaluated in the study done by Lira et al. [12] on the subject.

4.5 Experimental Results

We conducted a total of 4 experiments, obtaining results from three individual LLMs as well as the results from their ensemble. The LLMs utilized for this study are Meta’s Llama 3.1(8b version) [13], Anthropic’s Claude 3.5 Haiku, and Google’s Gemini 1.5 Flash.

Baseline: We will compare our results against the results obtained by Lira et al. [12], which show the performance of baseline LLMs for IDS systems as well as proposing a fine-tuned LLM model for this purpose. To the best of our knowledge, this is the state-of-the-art study for the use of LLMs for IDS. However, based on the results of our study, we wish to show that the use of baseline LLM models can achieve significant performance for IDS systems without going through the complicated and time-consuming process of fine-tuning an LLM. Fine-tuning LLMs is a time-consuming process, which, given the rate of improvement in this area, means that after all the effort put into fine-tuning a model, it becomes obsolete after a few months. Our system would enable the interchangeability of older models with newer ones. This would enable the use of better, improved models without the need to fine-tune them for use.

Table 4.5.1 contains the results obtained from our experiments. The first column indicates the name of the language model used, and the second column indicates the version of the model (if any). The remaining four columns indicate the different

Model name	Version	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
Llama	3.1 (8b)	89.31	88.24	85.19	86.69
Claude	3.5 Haiku	88.62	87.93	90.33	89.11
Gemini	1.5 Flash	87.79	88.43	89.16	88.8
Ensemble	-	91.32	89.65	90.51	90.08

Table 4.5.1: Results from experiments to evaluate proposed methodology on NSL-KDD dataset

evaluation metrics: Accuracy, Precision, Recall, and F1-score, respectively.

Looking at the result from Table 4.5.1, we achieved an accuracy of 91.32% using our ensemble method. As compared to the accuracy of 80% obtained by Lira et al on base LLMs, this shows a significant improvement in performance across all evaluation metrics. This shows that by combining the use of prompt optimization and ensemble methods, the performance of LLM-based IDS can be greatly enhanced. Moreover, when compared to the fine-tuning method proposed by Lira et al., our ensemble method achieved an accuracy of 91.32% and an F1-score of 90.08% while their model achieved an accuracy of 98.01% and an F1-score of 98.09%.

Table 4.5.2 shows the comparison between the two studies. While the fine-tuned model of the baseline paper shows better numerical results, the significance of our results lies in having a more advantageous framework when compared to the baseline while still maintaining promising performance. The previous authors have noted that fine-tuning LLMs comes at the risk of overfitting the model to a particular use case, and the subsequent generalization could lead to poor performance of the model when exposed to unseen data. In contrast to this, by using baseline LLMs, we eliminate the threat of overfitting since the LLMs are trained on data that is not specific to any particular domain.

Author	Method	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
Lira et. al [12]	Base LLM	80.0	-	-	-
	Fine-tuned LLM	98.01	98.31	98.01	98.09
Our method	Ensemble of LLMs	91.32	89.65	90.51	90.08

Table 4.5.2: Comparison with baseline research paper

4.5.1 Discussion

The key result noted from our experiments is that our approach of using prompt optimization and ensembling of LLMs outperforms the baseline LLM performance while maintaining comparable performance to Lira’s fine-tuned model. There are a few reasons why, despite Lira’s fine-tuning model performing relatively better, our approach has advantages over Lira’s method:

1. **Overfitting:** Given the massive amounts of training data pre-trained LLMs are trained on, they are not observed to be overfitted on any particular domain. However, when fine-tuning a pre-trained LLM, we train the model on a domain-specific dataset by freezing some layers of the neural network while updating the remaining layers. The updation is done by updating the model parameters according to the training set with stochastic gradient descent (SGD). Hence, this process involves careful tuning of hyperparameters such as learning rate and choosing which layers to freeze, which is why it is easy to go wrong and overfit on the training data. Moreover, due to the model complexity, the fine-tuning process involves smaller datasets, making the models more prone to memorizing specific patterns in the training set rather than generalized patterns.

With our approach, we eliminate the use of fine-tuning and modifying the model architecture. Instead, we use other techniques like prompt optimization and ensembling to achieve relatively good performance, especially when compared

to baseline pre-trained LLM. Due to the lack of fine-tuning, we can prevent overfitting altogether and ensure that the results are obtained from an unbiased model.

2. **Adaptability:** When working with LLM systems, adaptability is key. Since the release of ChatGPT in November of 2022, the pace of improvement in this domain has been rapid. Newer models quickly outperform older models in the span of a few months, to the point where older models become obsolete to the point of discontinuation in a matter of two years. This is why, despite spending resources to develop a well-performing fine-tuned model, these are unable to be upgraded with better models that are trained on newer data and, in some cases, outperform fine-tuned models. Doing so would require beginning the whole fine-tuning process from scratch for the newer LLM.

In our system, due to our modular approach of using pre-trained LLMs, upgrading the system is as simple as removing an older model and replacing it with a newer model. This ensures that the system can remain up-to-date with newer training data, detect newer and novel threats, and maintain a well-performing system over the years.

3. **Ease of development:** The process of fine-tuning a pre-trained LLM requires expertise in LLMs and their workings. Without it, it is easy to overfit the model to the training set. Moreover, due to the size and complexity of these models, the hardware requirement of this process is very high. Despite that, fine-tuning an LLM is a time-consuming process since models require multiple training iterations and extensive testing before deployment, taking weeks or months at a time.

In this study, since we used pre-trained models, developing this system is less knowledge-intensive, with a heavier emphasis on proper testing due to the lack of model training involved. Moreover, since we are only using an LLM rather than training one, the hardware and time requirements are significantly lower. Here, a system can be developed, tested, and deployed in a matter of days.

CHAPTER 5

Conclusion and Future Work

In this thesis, we introduced an optimized approach to the use of LLMs for IDS by prompt optimization methods and the ensembling of LLMs. Through our research of previous work in the area, we were able to identify shortcomings and gaps in existing research and propose a comprehensive solution to mitigate the problems. These problems included improper prompt optimization, complexity of the fine-tuning process, etc. Through our comprehensive evaluation, we were able to showcase the performance gains of our proposed methodology, including comparisons with a baseline model.

Across the different models evaluated, each demonstrated a higher performance than the baseline when optimizing their prompts. The combination of these LLMs for ensemble learning further enhanced the performance gains, and these results were even comparable to the state-of-the-art fine-tuned model proposed by Lira et al. while not requiring us to go through the complicated process of fine-tuning an LLM.

These results validated our hypothesis that by the use of prompt optimization and ensembling methods for LLMs, we can achieve significant performance without the process of fine-tuning. We noted that fine-tuning has many complications, such as the threat of overfitting, lack of upgradeability of the system with newer LLM models, and complexity of development. By using a more modular approach and enhancing the systems surrounding the LLM, we can build a well-performing IDS capable of detecting novel threats while maintaining upgradeability and capable of adapting to newer models trained on newer data with ease.

5.1 Future work

Looking ahead, there are some improvements that can be made to the framework. Firstly, future research can focus on the use of feature selection to improve performance. While there have been studies on feature selection on the NSL-KDD dataset, these studies focused on finding the best features to be used with older machine-learning models in order to obtain optimal performance. However, given the advanced nature of LLMs, existing feature selection may not be applicable to LLMs. Hence, better feature selection for LLMs may lead to performance gains.

Since this study was conducted on one particular dataset for a particular domain, this system can be improved by testing its applicability on other datasets and domains. Then, according to the findings on its performance in different domains, its shortcomings can be highlighted and the system can be further revised to suit other applications.

Further, more complex prompt optimization and ensembling methods could be applied to this system to improve performance. Newer optimization methods such as Textgrad [30] and Automatic Prompt Optimization (APO) [20] may prove to be helpful in increasing performance gains. For ensemble methods, instead of treating all models the same, model votes could be weighed differently by performing extensive testing to determine the better models. These models' predictions could be given a higher weight when obtaining a prediction from the ensemble.

REFERENCES

- [1] Abu-Ghazaleh, N., Ponomarev, D., and Evtyushkin, D. (2019). How the spectre and meltdown hacks really worked. *IEEE Spectrum*, 56(3):42–49. Conference Name: IEEE Spectrum.
- [2] Akuthota, V., Kasula, R., Sumona, S. T., Mohiuddin, M., Reza, M. T., and Rahman, M. M. (2023). Vulnerability Detection and Monitoring Using LLM. In *2023 IEEE 9th International Women in Engineering (WIE) Conference on Electrical and Computer Engineering (WIECON-ECE)*, pages 309–314. ISSN: 2837-8245.
- [3] Ali, T. and Kostakos, P. (2023). HuntGPT: Integrating Machine Learning-Based Anomaly Detection and Explainable AI with Large Language Models (LLMs). arXiv:2309.16021 [cs].
- [4] Almiani, M., AbuGhazleh, A., Al-Rahayfeh, A., Atiewi, S., and Razaque, A. (2020). Deep recurrent neural network for IoT intrusion detection system. *Simulation Modelling Practice and Theory*, 101:102031.
- [5] Alrawashdeh, K. and Purdy, C. (2016). Toward an Online Anomaly Intrusion Detection System Based on Deep Learning. In *2016 15th IEEE International Conference on Machine Learning and Applications (ICMLA)*, pages 195–200.
- [6] Alrzini, J. R. S. and Pennington, D. (2020). A review of polymorphic malware detection techniques. *International Journal of Advanced Research in Engineering and Technology*, 11(12):1238–1247. Num Pages: 305567 Number: 12.

- [7] Ariu, D., Tronci, R., and Giacinto, G. (2011). HMMPayL: An intrusion detection system based on Hidden Markov Models. *Computers & Security*, 30(4):221–241.
- [8] Chen, S.-S., Pai, T.-W., and Sun, C.-Y. (2023). Applying the Diamond Model of Intrusion Analysis with Generative Pre-trained Transformer 3. In *2023 International Conference on Consumer Electronics - Taiwan (ICCE-Taiwan)*, pages 289–290. ISSN: 2575-8284.
- [9] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805 [cs].
- [10] Du, M., Li, F., Zheng, G., and Srikumar, V. (2017). DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1285–1298, Dallas Texas USA. ACM.
- [11] Fang, C., Li, X., Fan, Z., Xu, J., Nag, K., Korpeoglu, E., Kumar, S., and Achan, K. (2024). LLM-Ensemble: Optimal Large Language Model Ensemble Method for E-commerce Product Attribute Value Extraction. arXiv:2403.00863 [cs].
- [12] G. Lira, O., Marroquin, A., and To, M. A. (2024). Harnessing the Advanced Capabilities of LLM for Adaptive Intrusion Detection Systems. In Barolli, L., editor, *Advanced Information Networking and Applications*, pages 453–464, Cham. Springer Nature Switzerland.
- [13] Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Sravankumar, A., Korenev, A., Hinsvark, A., ..., and Ma, Z. (2024). The Llama 3 Herd of Models. arXiv:2407.21783.
- [14] Jiang, D., Ren, X., and Lin, B. Y. (2023). LLM-Blender: Ensembling Large Language Models with Pairwise Ranking and Generative Fusion. arXiv:2306.02561 [cs].

- [15] Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T. T., Moazam, H., Miller, H., Zaharia, M., and Potts, C. (2023). DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. arXiv:2310.03714 [cs].
- [16] Le, V.-H. and Zhang, H. (2022). Log-based anomaly detection with deep learning: how far are we? In *Proceedings of the 44th International Conference on Software Engineering*, pages 1356–1367, Pittsburgh Pennsylvania. ACM.
- [17] Magazine, C. (2018). Cybercrime To Cost The World \$10.5 Trillion Annually By 2025.
- [18] Meng, W., Liu, Y., Zhu, Y., Zhang, S., Pei, D., Liu, Y., Chen, Y., Zhang, R., Tao, S., Sun, P., and Zhou, R. (2019). LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*, pages 4739–4745, Macao, China. International Joint Conferences on Artificial Intelligence Organization.
- [19] Naito, T., Watanabe, R., and Mitsunaga, T. (2023). LLM-based Attack Scenarios Generator with IT Asset Management and Vulnerability Information. In *2023 6th International Conference on Signal Processing and Information Security (ICSPIS)*, pages 99–103, Dubai, United Arab Emirates. IEEE.
- [20] Pryzant, R., Iter, D., Li, J., Lee, Y. T., Zhu, C., and Zeng, M. (2023). Automatic Prompt Optimization with "Gradient Descent" and Beam Search. arXiv:2305.03495 [cs].
- [21] Qi, J., Huang, S., Luan, Z., Yang, S., Fung, C., Yang, H., Qian, D., Shang, J., Xiao, Z., and Wu, Z. (2023). LogGPT: Exploring ChatGPT for Log-Based Anomaly Detection. In *2023 IEEE International Conference on High Performance Computing & Communications, Data Science & Systems, Smart City & Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys)*, pages 273–280.

- [22] Salvatore Stolfo, W. F. (1999). KDD Cup 1999 Data.
- [23] Singla, T., Anandayuvraj, D., Kalu, K. G., Schorlemmer, T. R., and Davis, J. C. (2023). An Empirical Study on Using Large Language Models to Analyze Software Supply Chain Security Failures. In *Proceedings of the 2023 Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*, SCORED '23, pages 5–15, New York, NY, USA. Association for Computing Machinery.
- [24] Tavallaei, M., Bagheri, E., Lu, W., and Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. In *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications*, pages 1–6. ISSN: 2329-6275.
- [25] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2023). Attention Is All You Need. arXiv:1706.03762 [cs].
- [26] Vishwakarma, S., Sharma, V., and Tiwari, A. (2017). An Intrusion Detection System using KNN-ACO Algorithm. *International Journal of Computer Applications*, 171(10):18–23.
- [27] Yilmaz, S. and ÇETER, M. (2018). Feature selection and comparison of classification algorithms for intrusion detection. *ANADOLU UNIVERSITY JOURNAL OF SCIENCE AND TECHNOLOGY A - Applied Sciences and Engineering*, 19:206–218.
- [28] Yin, C., Zhu, Y., Fei, J., and He, X. (2017). A Deep Learning Approach for Intrusion Detection Using Recurrent Neural Networks. *IEEE Access*, 5:21954–21961. Conference Name: IEEE Access.
- [29] Yuan, Y., Kaklamanos, G., and Hogrefe, D. (2016). A Novel Semi-Supervised Adaboost Technique for Network Anomaly Detection. In *Proceedings of the 19th ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems*, pages 111–114, Malta Malta. ACM.

- [30] Yuksekgonul, M., Bianchi, F., Boen, J., Liu, S., Huang, Z., Guestrin, C., and Zou, J. (2024). TextGrad: Automatic "Differentiation" via Text. arXiv:2406.07496 [cs].
- [31] Zhang, J., Zulkernine, M., and Haque, A. (2008). Random-Forests-Based Network Intrusion Detection Systems. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 38(5):649–659. Conference Name: IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews).
- [32] Zhang, X., Xu, Y., Lin, Q., Qiao, B., Zhang, H., Dang, Y., Xie, C., Yang, X., Cheng, Q., Li, Z., Chen, J., He, X., Yao, R., Lou, J.-G., Chintalapati, M., Shen, F., and Zhang, D. (2019). Robust log-based anomaly detection on unstable log data. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 807–817, Tallinn Estonia. ACM.
- [33] Ziems, N., Liu, G., Flanagan, J., and Jiang, M. (2023). Explaining Tree Model Decisions in Natural Language for Network Intrusion Detection. arXiv:2310.19658 [cs].

VITA AUCTORIS

NAME:	Mohammed S Ahmad
PLACE OF BIRTH:	Jeddah, Saudi Arabia
YEAR OF BIRTH:	1999
EDUCATION:	Jamia Millia Islamia, B.Tech in Computer Engineering, New Delhi, India, 2021 University of Windsor, M.Sc in Computer Science, Windsor, Ontario, 2025

ProQuest Number: 31838861

INFORMATION TO ALL USERS

The quality and completeness of this reproduction is dependent on the quality and completeness of the copy made available to ProQuest.



Distributed by
ProQuest LLC a part of Clarivate (2025).
Copyright of the Dissertation is held by the Author unless otherwise noted.

This work is protected against unauthorized copying under Title 17,
United States Code and other applicable copyright laws.

This work may be used in accordance with the terms of the Creative Commons license
or other rights statement, as indicated in the copyright statement or in the metadata
associated with this work. Unless otherwise specified in the copyright statement
or the metadata, all rights are reserved by the copyright holder.

ProQuest LLC
789 East Eisenhower Parkway
Ann Arbor, MI 48108 USA